ARDUINO PHYSICAL AI CHALLENGE INDIA 2026

Project Report

Organised by Robu.in × Arduino · Submit as PDF · Max 10 MB

Project Title	Edge DM- Edge AI Tabletop Game Master
Team Name	SomerSault
Registration / Team ID	APC-2026-TN-91865
Contest Track	Gaming, Robotics & Interactive AI
Institution & City	SRMIST, Chennai

Team Members (1–4)

Name	Email	Role
Team Leader	shivani.sarangi16050@gmail.com	
Member 2 (optional)	suppriya.s06@gmail.com	
Member 3 (optional)		
Member 4 (optional)		

1. Project Overview

Problem Statement

What real-world problem does your project solve? Who faces it? 2–5 sentences.

Traditional Tabletop Roleplaying Games (TTRPGs) generally rely on a human Game Master who has knowledge of several rulebooks and worldbuilding experience to be able to run the game for other players, creating a steep barrier of entry that leaves most potential players unable to play. Current digital solutions primarily rely on cloud-based LLMs that cannot be run locally, or interact with physical pieces, failing to give the traditional tabletop experience. **Edge-DM** solves this by acting as a locally run, edge-AI Game Master that can process natural voice input, track physical board pieces, and narrate in real time. This makes sure that tabletop game enthusiasts, casual gamers, and game shops can play fully immersive, plug-and-play game campaigns without needing a human DM or an internet connection.

How Your Project Works

Briefly explain your project and what it does. 100–200 words.

Edge-DM is an offline, Physical AI Dungeon Master running 100% locally on the Arduino UNO Q, utilizing the dual brain architecture of the Arduino UNO Q by merging AI gameplay and narration, computer vision and structured memory on its linux core.

1. **Local Setup and Web Interface:** At boot, the system narrates an opening world prompt over a speaker. Players connect access a web app hosted on the Qualcomm MPU through local WiFi to upload character sheets and set turn order.
2. **Real Time Voice Pipeline:** Players speak into a USB microphone which is put through Vosk STT on the MPU, passes context to a local Llama 3.2 (1B) model, and synthesizes audio narration using Piper TTS.
3. **Three Tier RAG Memory Engine:** the 32GB eMMC uses vector search to manage three layers of memory: **The Vault** – Permanent memory with rulebooks, **The Ledger** – complete dialogue logs, and **Short-Term Memory** – contains last 3 turns to maintain story context.
4. **System Prompts:** To maintain memory retention and rule adherence, the code contains multiple game relevant events with system prompts to enforce certain behavior and refer to permanent vault for instructions or information through the ledger. Each event has 15-20 words that triggers the prompts.
5. **Computer Vision Combat:** During combat, a top down OpenCV camera scans ArUco tagged game pieces, and calculates relative distances, attack ranges, and spell ranges.

Why Arduino UNO Q?

How does its on-device AI capability enable your solution?

The **Arduino UNO Q** is the only single board computer that provides the **dual brain architecture** we exactly required to run an offline, autonomous local AI Dungeon Master in real time.

- **Linux core with Qualcomm MPU:** Its 4GB RAM and 32GB eMMC storage natively host our full local AI stack namely, Vosk STT, Piper TTS, Llama 3.2 (1B), and an offline RAG memory database and python scripts helping us eliminate cloud latency, API costs, and internet dependency.
- **Real Time Linux Edge Vision:** OpenCV image processing is able to run top down directly on the Linux MPU to track ArUco-tagged miniatures, which helps the system calculate piece movement, spell ranges, and line of sight without without needing an external vision co processor.

2. Components Used (BOM)

List every component. The Arduino UNO Q purchase proof must be uploaded separately with your submission.

Component	Qty
Arduino UNO Q (ABX00173)	*1*
5V/3A USB-C Power Adapter Cord	1
USB Web Camera	1
External Speaker	1
External Mic	1
USB Data Hub	1
Sound Card	1

3. System Architecture & Circuit

Step-by-Step Workflow

Example: Sensor collects data → UNO Q runs AI model on-device → Output device acts.

Step-by-Step Workflow

- Step 1: Local Setup and Voice Capture
- Local Configuration: Upon boot, users get a greeting message and the Qualcomm MPU hosts a local web server over Wi-Fi. Players access the web app on their browsers to set character sheets and turn orders without external cloud dependencies.
- Voice Capture: Audio streams captured from an external mic run through alocal Vosk Speech-To-Text engine directly on the MPU.
- Step 2: Game Engine
- Speech-to-Text (STT): The MPU receives the raw audio stream and runs local Vosk STT to transcribe spoken player actions into text in real time.
- System Prompts and Retrieval Augmented Generation: The transcribed action is evaluated by a local Llama 3.2 (1B) LLM that is given to a thin flask server with no game logic merely six endpoints to warm up the language model into memory so that the user doesn't pay the model loading cost.
- This flask server gives the data to the main python file in which, an optimized Regex Intent Matrix runs and has 300+ trigger words that can be said by the players which are narrowed down to intents such as intent to begin combat, talk to an npc that trigger a system prompt relevant to this intent.
- The system prompts instruct to a) give a response relevant to the intent (such as commencing npc interaction) b) or check the Permanent vault for rules on how to behave (such as commencing skill checks if the player wants to do an action) and then narrate

words accordingly c) or look through the ledger to retrieve relevant information that the player asks for.

- A 3-tier database on the eMMC is used for performing these prompts correctly using RAG (Retreival Augmented Generation):

The Vault- is permanent memory consisting of the ttrpg rulebook and dungeon mastering guide,

The Ledger – stores the entire log of the game whenever a trigger word was said in game, and

Short-Term Memory - Memory of the last 3 turns to continually maintain relevant context in gameplay. These memories are parsed through using vector search.

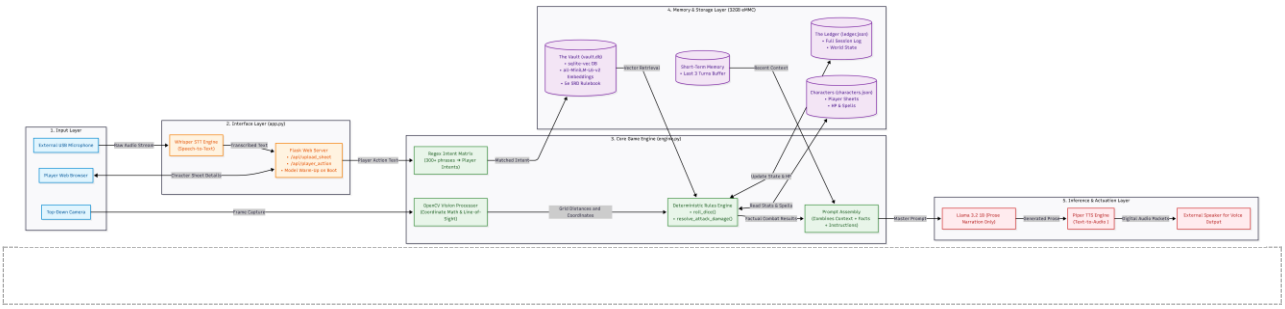
- The system also continually refers to the character sheets received from the webapp continually during turns as players update game relevant details like prepared spells to behave according to the updated character information. As the game progresses, data such as remaining HealthPoints of characters, if the characters are on death saves and previous npc interactions are deposited on the webapp without manual insertion.
- Computer Vision Spatial Math: Once combat mode is triggered, The Camera with OpenCV scans ArUco-tagged miniatures on the board, gives the data to the MPU. As a 1b model is inefficient in calculating combat math, we have opted to run it with python and the script runs the combat math that converts physical grid positions into coordinate data to calculate spell ranges, attack distances, and line-of-sight mechanics. This data is given to the AI to then narrate the happenings in game that occur due to the damages. This ensures smooth and autonomous gameplay for maximum immersion as the players don't have to spend time calculating the damage numbers.
- Narrative & Command Generation: Llama 3.2 synthesizes narrative text according to all the parameters described previously and feeds the text directly into Piper TTS for real-time speech generation.
- Step 3: Audio Narration
- Audio Playback: The MPU streams synthesized digital audio packets and outputs sound on the external speaker for voice narration and sound effects without audio stutter.

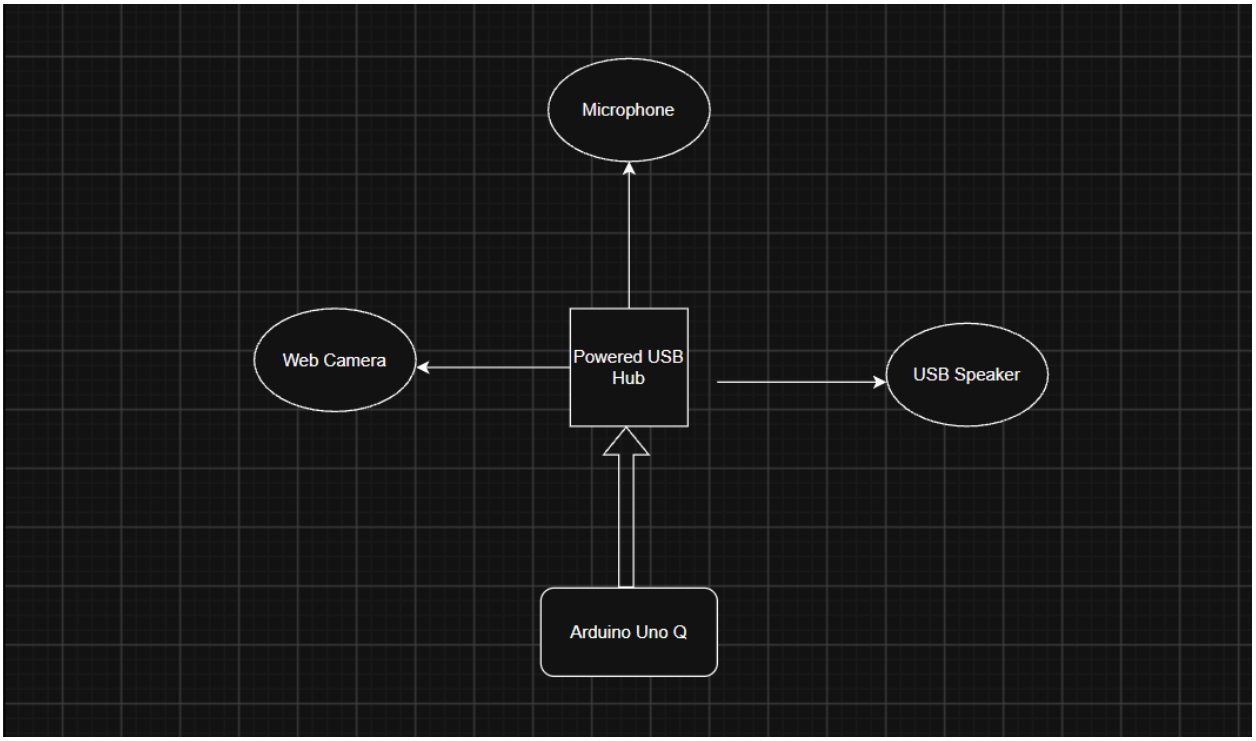
EXAMPLE OF ONE TURN END TO END

- A player says "I cast Ray of Frost at the goblin scout." Flask will receive that and and will hand it to the engine. The intent matrix correctly classifies it as combat due to the trigger word cast being said. The vault returns the relevant rules by vector similarity. Python looks up Ray of Frost on the character sheet, rolls 1d8 (the required dice roll for the attack), and subtracts the result from the goblin's hit points, and saves the new state. Only then is a prompt that tells the model what happened is created and asks it to describe it. The response is spoken aloud, and the exact numbers are returned separately in a combat_result field

Block Diagram & Circuit Schematic

Paste your block diagram and circuit/wiring diagram below. Hand-drawn photos, Fritzing, KiCad — all accepted. Label pin connections.





4. AI / ML Model Details

If your project does not use AI/ML, write "Not Applicable" — but note that Physical AI projects score higher on Innovation.

Model Used	*Llama 3.2 1B*
Training Platform	*NA- pretrained, not finetuned*
Accuracy	*NA, prose cannot be tested for accuracy.*

Dataset	*Dungeons and Dragons System Reference Document, Sly Flourish's Lazy GM Resource Document.*

Brief Description & Limitations

How does the model work in your project? When does it fail or degrade?

- For a 1-billion parameter model to behave as a storytelling game master without hallucinating math or memory about previous events, we used Retrieval Augmented Generation (RAG) through python scripts to give all the necessary information such as dice roll math and previous event information.
- A fast regex matrix classifies the intent (300+ phrases into 12 intents), while the exact rules are retrieved from the SQLite vector vault. Python's deterministic engine handles dice rolls, hit points, and ArUco combat ranges. The calculated results are injected into Llama 3.2 1B to synthesize natural narrative speech which is output via Piper TTS on a speaker.

---

5. Code Structure

Briefly explain the main functions. Example: setup() → initialises sensors; loop() → main execution; detectObject() → detection logic.

Code structure

The project is organized as six functional layers: offline knowledge preparation for the ai, game logic, computer vision tracking, the API layer, voice I/O, and deployment.

`build_vault.py` **— offline knowledge base builder (run once, before play)**

- `robust_read_json()` - goes through the SRD rulebook and Lazy DM's Guide JSON files, tolerating malformed or concatenated JSON. This is for providing in depth context.
- Main script body - embeds every rule via `TextEmbedding` (MiniLM-L6-v2) and writes the vectors plus source text into `vault.db` (SQLite + `sqlite-vec`), which `engine.py` queries at runtime.

aruco_tracker.py — overhead computer vision tracker (runs standalone alongside `app.py`**)**

- `find_grid_homography()` — calibrates the camera's view against 4 fixed corner ArUco markers, mapping pixel coordinates onto an 8×8 grid (5 ft per square, standard D&D combat scale).

- pixel_to_grid() — converts any detected marker's pixel position into a grid square using that calibration.
- main() — the tracking loop: detects every registered player/enemy marker each frame, converts positions to grid squares, and POSTs them to /api/update_positions every 1.5 seconds.

engine.py **— the game brain (**EdgeDMEngine **class)**

- detect_player_intent() - matches player speech against INTENT_MATRIX (12 intents, 300+ trigger phrases) via precompiled regex; returns which kind of action this is.
- get_vault_context() - embeds the player's sentence and retrieves the nearest matching rules from vault.db by vector distance.
- roll_dice() / resolve_attack_damage() - deterministically roll dice and apply damage to a target's HP in Python.
- find_spell_damage_dice() - looks up the caster's own prepared spells for exact damage dice before falling back to the rulebook.
- find_attack_range_ft() — reads the real range of the spell/attack being used from the caster's sheet, falling back to standard 5e defaults (5ft melee, 60ft generic ranged) if unset.
- update_board_positions() — stores the latest grid positions received from aruco_tracker.py.
- get_distance_ft() — computes real-world distance in feet between two tracked entities from their grid positions.
- check_attack_range() — gates combat on actual measured distance: if the camera has tracked both attacker and target, an out-of-range attack narrates as missing entirely and never rolls damage, the same "Python decides the facts, the LLM only narrates" principle used for dice math. Falls back to normal combat resolution if position data isn't available yet, so the system works identically with or without the camera running.
- build_master_prompt() - combines character sheet, retrieved rules, recent history, and an intent-specific instruction into one prompt, and returns the pre-computed combat_result alongside it.
- generate_narration() - sends the assembled prompt to the local Llama 3.2 1B model via Ollama (127.0.0.1:11434) and returns its response.
- get_turn_order() / get_current_turn_player_id() / advance_turn() - track whose turn it is based on the playOrder field set on the website, and rotate automatically after each action.
- generate_opening_narration() - produces a personalized session intro seeded with random world/threat/hook combinations so the LLM doesn't repeat the same setting.
- save_state() / _load_json() - persist and reload ledger.json (transcript, world state) and characters.json (all character sheets) so sessions survive restarts.

app.py **— Flask API layer**

- upload_sheet(), update_stat(), update_ledger() - receive character sheet and world state data from the website.
- session_status() - reports whether the game has begun and who's joined, for the website's waiting screen.
- start_session() - triggers the personalized opening narration.
- current_turn() - returns whose turn it is, polled by voice_loop.py before each recording.
- update_positions() — receives grid positions from aruco_tracker.py and stores them via EdgeDMEngine.update_board_positions().
- handle_player_action() - the main gameplay endpoint; calls EdgeDMEngine.process_player_action() and returns narration plus the exact combat_result numbers.
- warmup_model() (background thread, started at import) - loads the LLM into memory at server startup so the first real turn isn't slowed by a cold load.

voice_loop.py **— voice interface (runs standalone alongside** app.py**)**

- record_action() - captures microphone audio via arecord for a fixed window.
- transcribe() - offline speech-to-text via Vosk.
- speak() - offline text-to-speech via Piper, piped to the USB sound card.
- get_current_turn() - polls /api/current_turn so the loop always addresses the correct player by name, with no hardcoded player ID.
- main() - the turn loop: wait for discussion time then prompt the current player by name then record then transcribe then POST to /api/player_action then speak the reply then repeat until an end-session phrase is detected.

start.sh **/** dm-engine.service **/** setup_audio.sh **— deployment**

- setup_audio.sh installs all voice dependencies (Vosk, Piper, the speech model) in one step.
- start.sh launches app.py, waits for the model to finish warming up, starts aruco_tracker.py in the background, then launches voice_loop.py — bringing up the full sense-think-act pipeline with one command.
- dm-engine.service is a systemd unit for automatic startup on boot.

GitHub Repository	*https://github.com/Suppriya-Sree-J/Edge-AI-TTRPG-Dungeon-Master*
Demo Video Link	*https://youtu.be/YpaCznJ4X1w*

6. Testing & Results

How did you test the complete system? What were the results? Include accuracy, response time, edge cases.

Methodology & Testing Setup

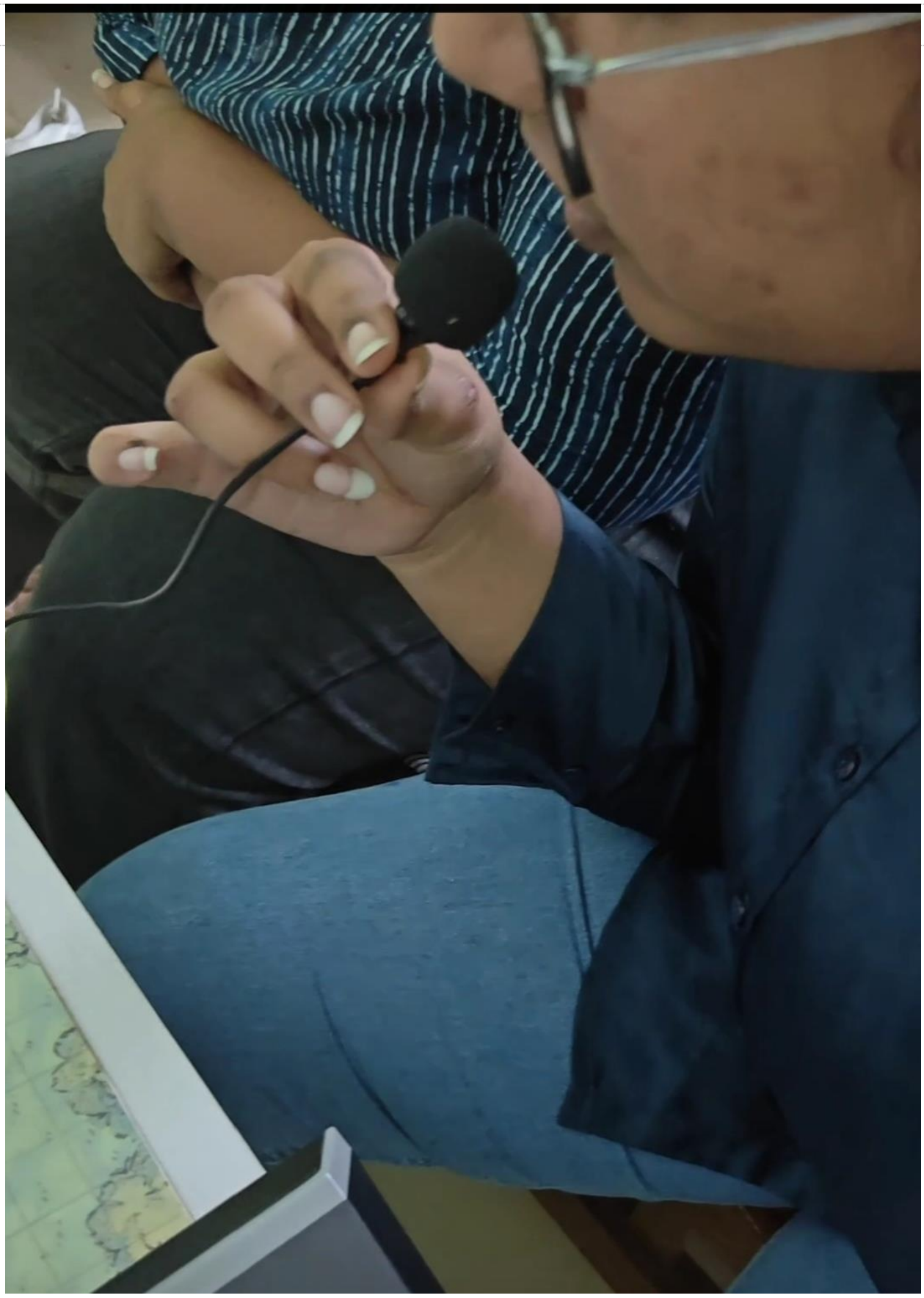
- **Memory & Text Retrieval:** Evaluated context retrieval speed and relevance by executing natural language dialogue queries (e.g., asking to recall past NPC dialogue or specific D&D 5e SRD spell rules).
- **Board Vision & Combat Processing:** Tested board scanning using an overhead camera tracking ArUco markers. Tested spatial Euclidean distance calculations and automated HP/dice resolution via Python.
- **System End-to-End Test:** Ran turn-based gameplay loops connecting the Flask API, intent detection engine, vector/FTS database, and local Llama 3.2 1B model via Ollama.

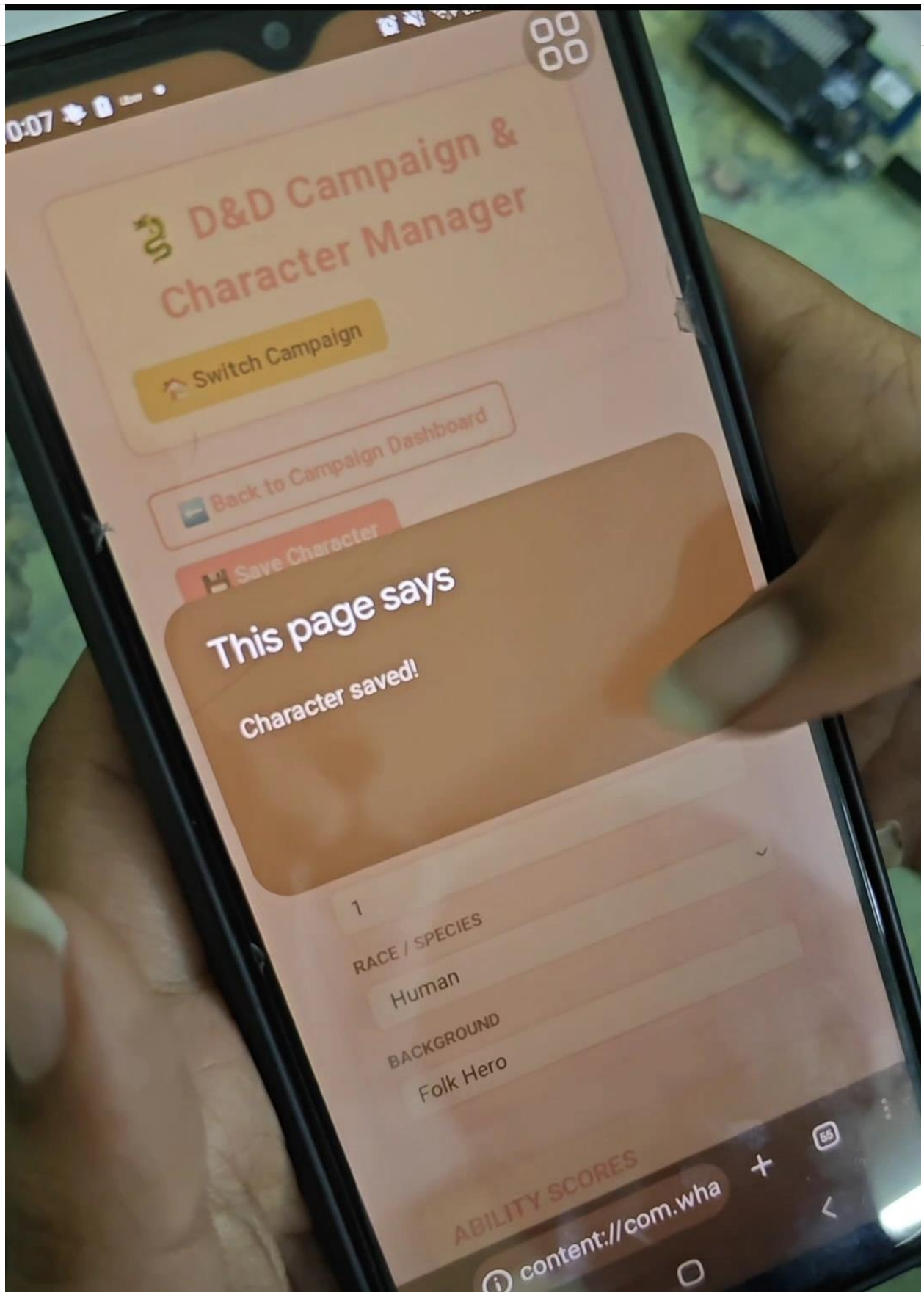
Edge Cases & Resolutions

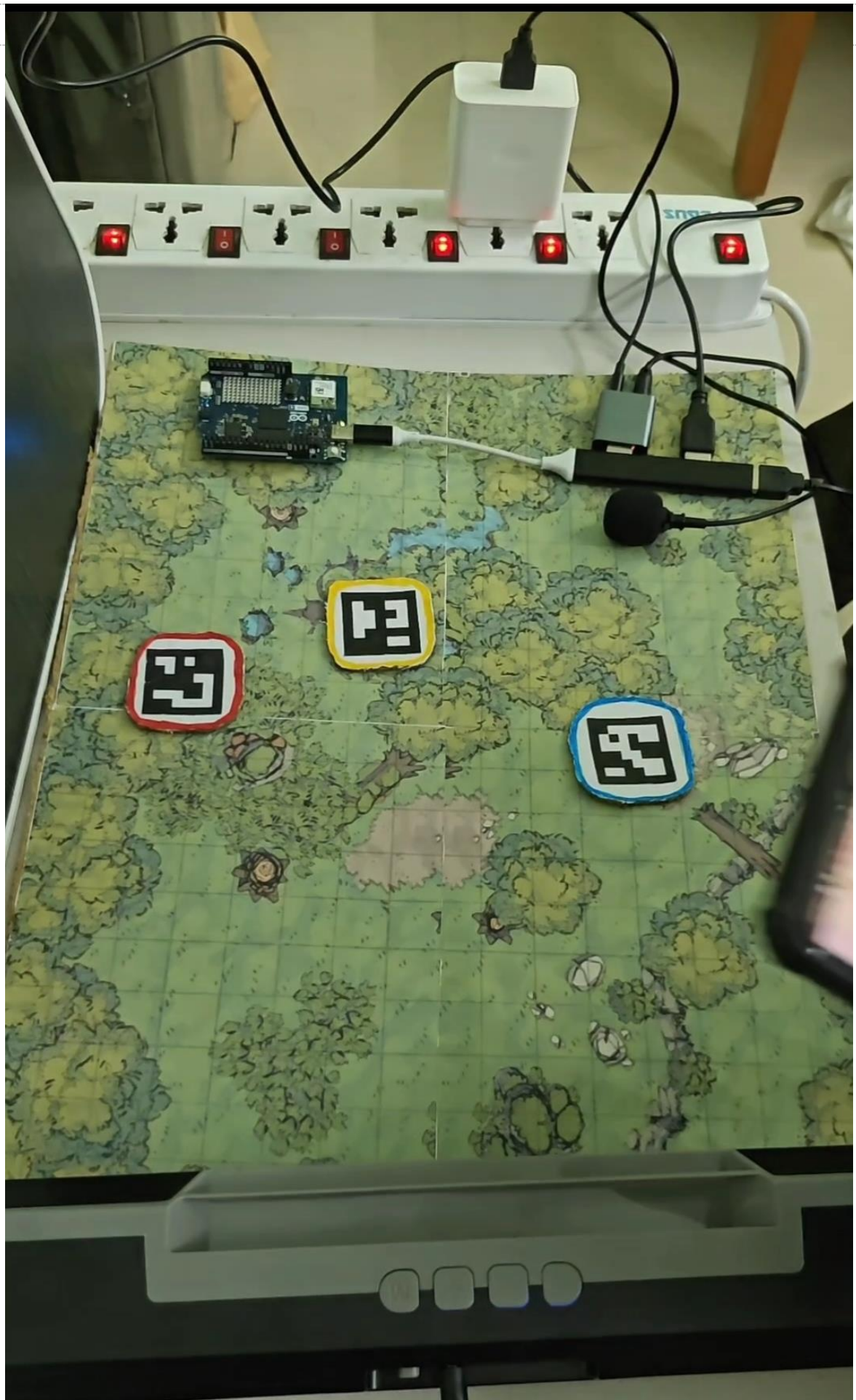
- **Edge Case: Arithmetic Hallucinations in LLM**
  - *Issue:* The 1B parameter model hallucinated incorrect HP totals and fabricated enemy counter-attacks during initial tests.
  - *Resolution:* Offloaded all mathematical and combat calculations entirely to Python (`engine.py`); the LLM was restricted strictly to generating descriptive prose based on structured JSON data.

Project Images (2–3)

[Insert 2–3 photos of your project working]




D&D Campaign & Character Manager
Switch Campaign
Back to Campaign Dashboard
Save Character
This page says
Character saved!
1
RACE / SPECIES
Human
BACKGROUND
Folk Hero
ABILITY SCORES
content://com.wha



7. Challenges, Learnings & Future Improvements

Challenges Faced

e.g. sensor calibration, model optimisation, power management.

- Challenges Faced
- LLM Arithmetic Hallucinations & Math Reliability: When we tested it initially, we found out that Llama 3.2 1B frequently made errors on dice calculations, inverted hit point logic, and hallucinated the damage numbers. The combat mode performed no differently than out of combat narration/adventuring.
- To fix this, We stripped all rule and damage calculations from the LLM. Python (engine.py) handles all of the game math deterministically, passing raw facts to the LLM so that it could do the narration and storytelling seamlessly.
- On-Device Memory & Buffer Overhead: We figured that running continuous OpenCV camera streams alongside our local LLM models risked causing Out-Of-Memory (OOM) issues. Mitigation: We made sure the camera only runs when the players are in combat mode.

  First-Turn Latency Overhead: Cold-starting the LLM caused delays on the first turn. So, we Integrated a thin flask server deliberately containing no game logic (app.py). It serves as the web interface and has six endpoints. This warms up the LLM into memory at startup so that players don't have to pay the model loading cost.

What You Learned & What's Next

Summarise what you built, what you learned, and 2–3 future improvements.

- We built Edge-DM, an offline physical AI Dungeon Master that runs storytelling Table Top Role Playing Games, 100% locally on the Arduino UNO Q.
- The board's dual-brain architecture, helps us combine the AI local generative prose (Llama 3.2 1B) and use python scripts on its 32 GB eMMC memory to run a RAG system that performs seamlessly as a game master without hallucinating. The board's powerful 4gb MPU also helped us integrate a speech to text(Vosk) and text to speech(Piper) pipeline directly on the MPU unit to deliver immersive gameplay. We also built automated combat calculation through arUco tag detection through OpenCV computer vision along. A local webapp is hosted on the board's local WiFi to provide character sheet details for the narrative, it has two way communication as the system automatically updates details such as Hit Points on the webapp without manual intervention.

- What We Learned:
- We learned that small 1-billion-parameter edge models cannot be trusted with arithmetic or complex rule enforcement and so, the key to making physical AI reliable is strict separation of concerns. Python handles all of state tracking, dice rolls, and coordinate math, leaving the LLM to focus purely on generating narrative, evolving prose.
- Regex Pre-Filtering for RAG: Running vector search across massive databases introduced latency overhead. So, we pre filtered user input with a fast, regex compiled INTENT MATRIX of 300+ phrases mapped to 12 intents which decreases the RAG retrieval time massively.
- Resource Constraints on Edge Hardware: Running continuous camera streams alongside LLMs can easily trigger Out-Of-Memory (OOM) crashes, therefore we limited the camera runtime to remain within combat mode.

Future Improvements

Expand the STM32 MCU's RPC bridge to dynamically control ambient room lighting and localized audio effects tied to live game states (e.g., flashing red lights during combat rounds).

Develop a physical dice tray sensor, to make the gameplay flow even more seamlessly and autonomously.

AI finetuned and trained with LoRA adapters made specifically for Dnd, currently we are using it only for combat context.

---

8. Declaration

We confirm this is our original, unpublished work. The Arduino® UNO™ Q is the primary board in this project. All team members have reviewed and agree to this report.

Date	25/8/26